
MatToolBench

Mei Wu^{1,2*} Rui Xie^{1*} Runyu Zhang¹ Lu Chen^{1,2†} Bo Chen²
Kai Yu^{1,2†} Xin Chen^{2†}

¹X-LANCE Lab, Department of Computer Science and Engineering,
MoE Key Lab of Artificial Intelligence, SJTU AI Institute,
Shanghai Jiao Tong University, Shanghai, China
²Suzhou Laboratory, Suzhou, China

Abstract

Multimodal GUI agents have achieved impressive results on general software benchmarks (e.g., 72.1% success rate on OSWorld), yet their ability to operate professional scientific software remains largely unexplored. We present **MatToolBench**, the first real-environment benchmark for evaluating multimodal GUI agents on professional materials science software. MatToolBench comprises **204 tasks** across **10 tools**, covering three interaction modalities: GUI operation (JADE, VESTA, Avantage, DigitalMicrograph, Materials Studio), OriginPro scripting, and code-based database queries (MP, OQMD, OPTIMADE, Pymatgen).

To accurately measure agent performance in this domain, we design a multi-level evaluation framework combining text-based OCR matching, visual-text hybrid assessment, LLM-based aesthetic judgment, and code matching. Evaluation criteria are decomposed into fine-grained sub-criteria by materials science experts, yielding highly interpretable partial-credit scores and dense reward signals suitable for reinforcement learning training. The framework achieves an average evaluator F1 of 0.98 across all GUI domains.

We evaluate seven state-of-the-art multimodal models spanning four commercial providers. Despite their strong general capabilities, the best models achieve only **25% SR on GUI tasks** and **45% SR on code tasks**—substantially below their performance on general benchmarks. Analysis reveals key failure modes: MS (Materials Studio) is the hardest GUI domain (<20% SR for most models), OPTIMADE tasks are most tractable (up to 85% SR), and domain-specific prompt templates provide consistent gains especially for GUI tasks (e.g., removing hints drops Avantage SR from 35% to 20%). MatToolBench thus serves both as a challenging diagnostic for current agent capabilities and as an RL-ready training environment for advancing GUI agents in scientific workflows.

1 Introduction

With the rapid development of multimodal large language models and the extensive application of supervised fine-tuning (SFT) and reinforcement learning (RL) on general graphical user interfaces (GUIs), GUI agents have achieved remarkable progress in visual perception, decision-making, and grounding capabilities [OpenAI \[2024\]](#), [Hong et al. \[2024\]](#). They have demonstrated strong performance in general-purpose scenarios [Xie et al. \[2024\]](#), [Zhou et al. \[2024\]](#), such as web browsing [Zhou et al. \[2024\]](#), desktop operation [Xie et al. \[2024\]](#), or synthetic environments [Shridhar et al. \[2021\]](#). However, existing research remains predominantly confined to everyday general-purpose software.

*Equal contribution.

†Lu Chen, Kai Yu, and Xin Chen are the corresponding authors.

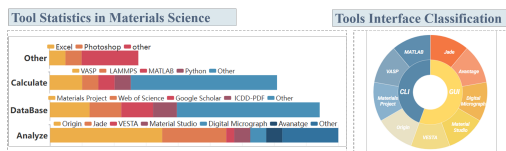


Figure 1: Survey results from 25 materials researchers: tool usage frequency across sub-fields (left) and distribution of primary interaction interface types (right).

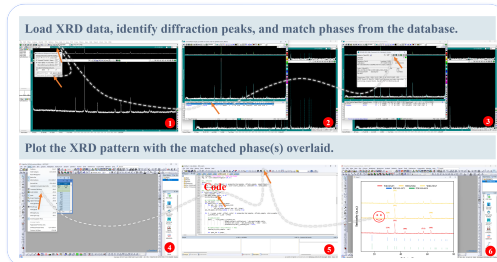


Figure 2: A complete XRD diffraction pattern requires cross-tool collaboration: raw XRD data is analyzed in JADE, then plotted and annotated in OriginPro.

Due to the scarcity of web data related to professional scientific software, the pre-trained knowledge of large models in this vertical domain is inevitably constrained, leaving it an open question whether they can exhibit similarly excellent performance in complex, real-world scientific workflows. Materials chemistry tools, characterized by complex graphical interfaces and strong domain dependencies, offer an ideal platform for systematically testing the domain-specific capabilities of multimodal GUI agents.

Compared to general-purpose scenarios, the use of materials tools exhibits distinctive complexity: (1) Highly fragmented tool ecosystem. As shown in Figure 1, a survey of materials science graduate students reveals that different research subfields and usage requirements correspond to a large number of heterogeneous tools, which often belong to different operating environments and interaction interfaces. (2) Highly cross-tool task workflows. As shown in Figure 2, completing a full materials analysis task typically requires frequent switching and coordination across multiple tools. (3) Severely limited accessibility interfaces. Such professional GUI software commonly features information-dense interfaces, deeply nested menu hierarchies, and implicit feedback mechanisms for critical operation states, lacking unified and standardized accessibility interfaces.

We present **MatToolBench**, a real-environment benchmark for evaluating multimodal GUI agents on professional materials science software. It covers 10 tools and 204 tasks across GUI, code, and cross-tool interaction modalities, and pairs them with a multi-level evaluation framework—text-based OCR matching, visual-text hybrid assessment, LLM aesthetic judgment, and code matching—all grounded in fine-grained, expert-designed scoring sub-criteria. GUI evaluators achieve an average F1 of 0.98 (JADE 0.991, MS 0.994, Avantage 0.980, VESTA 0.979; see Appendix Table 8), and the decomposed sub-criteria provide dense RL reward signals for future training.

2 Related Work

2.1 GUI-based Agents

GUI agent research has progressed through a gradual evolution from web and mobile to general desktop operating systems. WebArena [Zhou et al. \[2024\]](#) and Mind2Web [Deng et al. \[2023\]](#) established benchmarks for web browsing tasks, while AITW [Rawles et al. \[2023\]](#) focused on mobile GUI interaction. On the desktop side, OSWorld [Xie et al. \[2024\]](#) introduced a cross-platform general benchmark, and WindowsAgentArena [Bonatti et al. \[2025\]](#) further enhanced desktop perception through the Windows Accessibility Tree. Although ProSoftArena [Ai et al. \[2026\]](#) extended evaluation to professional creative software such as Adobe and Figma, its task objectives remain within the scope of general operations, and such software benefits from abundant interaction data and tutorials on the internet.

Existing benchmarks have not yet addressed the GUI ecosystem of professional scientific domains. Unlike general-purpose or creative software, professional software in fields like materials science presents a dual challenge. First, due to its highly vertical nature, relevant interaction data and operational demonstrations on the web are extremely scarce, resulting in natural “knowledge blind spots” for general agents that rely on large-scale pre-training. Second, the operational logic of these

tools extends far beyond basic interface interaction, deeply coupling with complex, domain-specific scientific knowledge. Consequently, existing benchmarks fail to measure the true performance of agents in data-scarce, knowledge-intensive real-world scientific workflows.

2.2 Science Benchmarks

Agent evaluation in the scientific domain has undergone a paradigm shift from knowledge reasoning to program execution. SciBench Wang et al. [2024] and MatSciBench Zhang et al. [2025] primarily rely on textual reasoning; ScienceAgentBench Chen et al. [2025], SciCode Tian et al. [2024], and MatTools Liu et al. [2025] further incorporate code execution; while Spider2-V Cao et al. [2024], ScienceBoard Sun et al. [2026], and LabBench Laurent et al. [2024] extend evaluation to multi-step workflows spanning multiple tools. However, all the aforementioned benchmarks evaluate agents exclusively through structured APIs, program code, or backend logs, without requiring genuine graphical interface perception or interaction capabilities.

Although works such as CLI-Anything attempt to invoke software functionalities via command-line interfaces (CLIs), the professional tools widely used in scientific research (e.g., VESTA, Materials Studio, or Jade) inherently rely on complex 3D rendering, high-density information charts, and implicit visual feedback. These interactive dimensions fundamentally cannot be encapsulated or replaced by pure text or CLI interfaces. To bridge this critical gap, this work presents the first comprehensive multimodal agent benchmark for materials science, requiring models not only to possess scientific reasoning capabilities but also to execute professional software operations and cross-tool collaboration within realistic GUI environments.

3 MatToolBench Environment

As illustrated in Figure 3, the MATTOOLBENCH environment is built on a Windows 11 virtual machine and exposes three interaction modalities—GUI operation, Python code generation, and OriginPro script authoring—through a unified Gymnasium interface.

3.1 Task Definition

Task execution is formalized as a POMDP $\langle \mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{T}, \mathcal{R} \rangle$. At each step t the agent receives observation o_t (screenshot/execution feedback), takes action a_t , and transitions to s_{t+1} . Policy π conditions on the full history $m_t = \{g, o_0, a_0, \dots, o_t\}$ where g is the fixed natural-language instruction. Episodes terminate on: DONE (agent declares success), FAIL (agent declares infeasible), or step-budget exhaustion (default 50 for GUI/Origin, 10 for code). A WAIT action is available for slow GUI responses.

Reward Design for RL. A central design goal of MATTOOLBENCH is to serve as an *RL-friendly* training environment. Rather than providing only a binary success/failure signal, each task is decomposed into multiple **scoring sub-criteria** (detailed in Section 4.1), each corresponding to a verifiable intermediate or final state the agent must achieve. Upon episode termination, the task-specific evaluator independently assesses every sub-criterion and produces a fine-grained score vector, from which the overall task score is computed as the fraction of sub-criteria satisfied. This decomposed reward structure yields substantially denser feedback than a single binary outcome: it tells the agent *which* sub-goals were achieved and which were missed, enabling more informative credit assignment during policy optimization. For tasks annotated as infeasible, the agent receives a score of 1 only if it correctly emits FAIL.

3.2 Observation Space

GUI-Based Tasks. Each observation o_t for desktop-application tasks is a configurable tuple comprising: the task instruction g ; a full-desktop screenshot v_t (1920×1080); an optional UIA accessibility tree e_t ; an optional Set-of-Marks (SoM) image r_t in which UI regions from e_t , OmniParser Lu et al. [2025], or both are overlaid as numbered bounding boxes; foreground window metadata; and clipboard content. Six `observation_type` modes are supported, ranging from raw screenshot only (`no_omni`) to accessibility + OmniParser hybrid (`mixed-omni`).

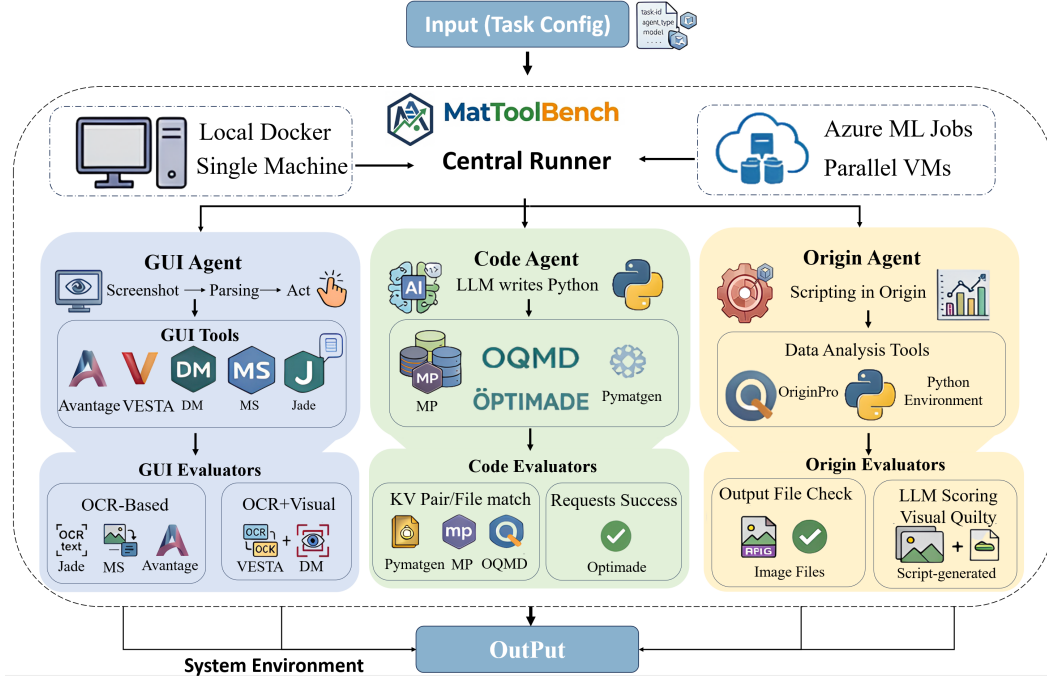


Figure 3: Overview of the MATTOOLBENCH framework. Tasks are dispatched by a central runner to one of three specialized agents: the *GUI Agent* operates materials science desktop applications (e.g., JADE, VESTA, Avantage) via screenshot parsing and element grounding; the *Code Agent* generates Python scripts to query materials database APIs (e.g., MP, OQMD, OPTIMADE); and the *Origin Agent* combines GUI interaction with OriginPro script generation for experimental data plotting. All agents execute inside a Windows 11 VM hosted within a Docker container, communicating through a Flask-based control server. Task-specific evaluators assess the final environment state after each episode.

Code and Origin Tasks. For database tasks (MP, OQMD, OPTIMADE, Pymatgen), o_t reduces to text-only execution feedback (stdout, stderr, return code); no visual signal is provided. For Origin tasks, observations additionally include the current OriginPro workspace screenshot, enabling the agent to verify whether the generated plot meets requirements.

3.3 Action Space

GUI Actions. GUI tasks use a hierarchical `Computer` interface with five sub-modules: `Mouse` (element-ID or normalized-coordinate positioning, click, scroll, drag), `Keyboard` (character input and hotkeys), `Clipboard` (text/image copy-paste), `OS` (program launch), and `WindowManager` (fuzzy-match application switching). Actions are transmitted as Python code strings executed via `exec()` on the VM; the element-ID-based `move_id()` reduces sensitivity to localization errors. The full API is in Appendix Table 4.

Code and Origin Actions. Code tasks reduce to a generate-and-correct loop: the agent writes a complete Python script per step, executed in an isolated virtual environment per domain; stdout/stderr/return-code are fed back for self-correction. Origin tasks inject scripts into OriginPro’s Code Builder via clipboard paste and F5, combining code generation with GUI navigation.

3.4 Execution Environment

The environment has two layers: an **outer** Linux Docker container for agent logic and visual parsing (OmniParser), and an **inner** Windows 11 VM on QEMU/KVM (1920×1080) pre-installed with all target applications and isolated Python venvs per code domain. A Flask HTTP server in the VM (port 5000) is the sole control interface; external API calls route through a tinyproxy instance in the

Table 1: Benchmark task statistics. E/M/H: easy / medium / hard (1–3 / 4 / 5–7 pts).

Cat.	Domain	#	E/M/H	Avg	Tot.
GUI	Avantage	20	13/2/5	3.75	75
	DM	20	13/4/3	3.35	67
	JADE	20	14/3/3	3.45	69
	MS	20	6/6/8	4.10	82
	VESTA	20	13/2/5	3.45	69
Origin	Origin	16	—	2.00	32
Code	MP / OQMD / OPTIMADE / Pymatgen	80	—	1.00	80
Mixed	Mixed	8	—	—	—
Total		204			394+



Figure 4: Task distribution across all 10 domains and sub-categories. The inner ring shows domains; the outer ring shows sub-categories with their percentage of total tasks.

container. Each episode resets by closing applications and running task-specific JSON setup scripts (file download, app launch, window activation). The framework exposes a standard Gymnasium interface (`reset()`, `step()`, `evaluate()`) and supports concurrent multi-worker evaluation.

4 The MATTOOLBENCH Benchmark

4.1 Task Statistics

MATTOOLBENCH comprises **204 tasks** spanning **10 domain-specific tools**, organized into three agent types: GUI-based (JADE, Avantage, VESTA, DM, MS), Origin-based (Origin), and code-based (MP, OQMD, OPTIMADE, Pymatgen, VASP). Figure 4 illustrates the sub-category distribution within each domain.

Scoring Scheme. Each task is decomposed into one or more **scoring sub-criteria** (pts), each corresponding to a distinct intermediate or final state the agent must correctly achieve. The per-task score equals the fraction of sub-criteria satisfied, enabling partial credit on multi-step tasks. A task is counted as a full success (SR = 1) only when all sub-criteria are met.

Difficulty Levels. Tasks are stratified into three levels based on the number of scoring sub-criteria, which serves as a proxy for operational complexity: **Easy** (1–3 pts) involves single-step or few-step operations such as file loading or single-function execution; **Medium** (4 pts) requires chained multi-step workflows with intermediate verification; and **Hard** (5–7 pts) demands complex workflows combining multiple functions, parameter configuration, and end-to-end result validation.

4.2 Evaluation Protocol

Evaluation follows a two-stage pipeline after each episode: a domain-specific **getter** extracts the task-relevant result, and a **metric function** scores it against a reference. GUI results are extracted via OCR or pixel analysis; code results are written to files and compared against a gold-standard directory; Origin results use file-existence check plus an optional GPT-4o visual judge. Metric details are in Appendix Tables 5–6.

Metrics. We report **Score** (normalized fraction of sub-criteria satisfied, enabling partial credit) and **SR** (success rate: all sub-criteria met). Efficiency is captured by **Eff*** (valid-action ratio on successful episodes) and **FTR** (false termination rate: fraction of agent-terminated episodes where SR = 0), both computed over agent-terminated episodes only.

Table 2: Accuracy results on MATTOOLBENCH. Sc. (Score): normalized task score (%) averaged over sub-criteria; SR: success rate (%; all sub-criteria satisfied). **Bold**: best per metric; underline: second best. Abbrev.: doubao seed-1.8 = doubao-seed-1-8; Claude son.-4.6 = Claude-sonnet-4.6; Qwen3 235B/8B = qwen3-vl-235b/8b; Qwen 32B = qwen-vl-32b.

Cat.	Domain	doubao seed-1.8		kimi k2.5		Claude son.-4.6		GPT -5.4		Qwen3 235B		Qwen 32B		Qwen3 8B	
		Sc.	SR	Sc.	SR	Sc.	SR	Sc.	SR	Sc.	SR	Sc.	SR	Sc.	SR
GUI	Avantage	44.6	35.0	32.3	<u>20.0</u>	<u>41.3</u>	35.0	32.3	15.0	30.8	10.0	26.2	15.0	18.5	15.0
	JADE	<u>54.2</u>	25.0	50.8	25.0	57.6	25.0	50.8	<u>20.0</u>	45.8	10.0	37.3	10.0	37.3	5.0
	DM	56.7	20.0	<u>52.2</u>	20.0	40.3	20.0	50.7	20.0	<u>52.2</u>	<u>10.0</u>	6.0	0.0	38.8	<u>10.0</u>
	MS	47.6	15.0	35.4	10.0	<u>52.4</u>	<u>20.0</u>	56.1	30.0	43.9	0.0	26.8	0.0	20.7	0.0
	VESTA	52.2	<u>20.0</u>	<u>59.4</u>	25.0	58.0	25.0	65.2	<u>20.0</u>	52.2	10.0	29.0	10.0	30.4	10.0
	Avg.	52.1	<u>24.0</u>	46.0	20.0	49.9	25.0	<u>51.0</u>	21.0	45.0	8.0	25.1	7.0	29.1	8.0
Origin	(pending)	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Code	Pymatgen	30.0	30.0	15.0	15.0	25.0	25.0	35.0	35.0	35.0	35.0	<u>29.4</u>	<u>29.4</u>	15.0	15.0
	MP	25.0	25.0	<u>22.5</u>	<u>20.0</u>	21.9	15.0	17.5	15.0	25.0	25.0	10.0	10.0	10.0	10.0
	OQMD	33.2	10.0	50.3	30.0	60.8	50.0	58.2	<u>45.0</u>	26.2	10.0	11.5	0.0	18.4	0.0
	OPTIMADE	70.0	70.0	<u>80.0</u>	<u>80.0</u>	75.0	75.0	85.0	85.0	20.0	20.0	30.0	30.0	10.0	10.0
	Avg.	39.6	33.8	42.0	36.2	<u>45.7</u>	<u>41.3</u>	48.9	45.0	26.6	22.5	20.2	17.4	13.4	8.8
Overall Avg.		—	—	—	—	—	—	—	—	—	—	—	—	—	—

5 Experiments

5.1 Experimental Setup

Models. We evaluate seven state-of-the-art multimodal models spanning three commercial providers: **doubao-seed-1-8** (ByteDance), **kimi-k2.5** (Moonshot), **Claude-sonnet-4.6** (Anthropic), **GPT-5.4** (OpenAI), **qwen3-vl-235b** and **qwen-vl-32b** (Alibaba), and **qwen3-vl-8b** (Alibaba). All models are accessed via their respective official APIs. For GUI and Origin tasks, each model is wrapped in the NAVIAGENT framework with SoM-annotated screenshots (mixed-oss mode) and a system prompt tailored to the agent type; for code tasks, no visual input is provided. The maximum step budget is 50 for GUI/Origin tasks and 10 for code tasks.

Claude Coordinate Calibration. During evaluation, we identified an important model-specific behavior in Claude-sonnet-4.6: although screenshots are captured and transmitted at the native 1920×1080 resolution, Claude internally resizes images to 1456×816 pixels before processing. Without explicit correction, the model computes normalized click coordinates by dividing pixel positions by 1920 and 1080, yielding systematically mis-calibrated coordinates (a deviation of ≈24% horizontally and ≈24% vertically). This causes nearly all element-level interactions to miss their targets, resulting in a dramatic drop in task performance. To address this, we inject the following calibration notice at the start of every Claude system prompt:

```
[CLAUDE_COORD_RULE_ACTIVATED: 1456x816]
Claude internally resizes images to 1456 × 816px.
Use normalized_x = pixel_x / 1456, normalized_y = pixel_y / 816.
Do NOT divide by 1920 or 1080.
```

Without this correction, Claude-sonnet-4.6 achieves near-zero GUI success rates across all domains. With it, the model recovers to competitive performance, as reported in Table 2. We emphasize that this is an inherent property of the Claude model family’s vision pipeline and is not specific to our benchmark; any GUI benchmark that requires precise coordinate-level interactions should apply the same correction when evaluating Claude models.

5.2 Main Results

Table 2 reports Score and SR for all seven models across GUI, Origin, and Code domains. The overall performance level is consistently low, with the best-performing models achieving only 24–25% SR on GUI tasks and 41–45% SR on code tasks, highlighting the substantial difficulty of professional materials science workflows relative to general GUI benchmarks.

GUI Tasks. The top performers on GUI tasks are doubao-seed-1-8 and GPT-5.4, achieving average SRs of 24.0% and 21.0% respectively. Claude-sonnet-4.6 ranks second overall with an average SR of 25.0%, benefiting from strong visual grounding once the coordinate calibration (Section 5.1) is applied. Models without adequate GUI grounding — notably qwen-vl-32b (7.0% avg. SR) — struggle on tasks requiring precise multi-step interaction sequences.

Among individual domains, **MS** (Materials Studio) is consistently the most challenging, with all models except GPT-5.4 scoring below 20% SR. This reflects Materials Studio’s highly nested menu hierarchy and implicit feedback after operations. In contrast, **VESTA** and **JADE** show relatively higher scores, likely because these tools provide more explicit visual confirmation of state changes.

A notable gap exists between Score and SR across all GUI domains: models routinely complete a fraction of scoring sub-criteria (e.g., Score $\approx$ 50%) while rarely satisfying all of them simultaneously (SR $\approx$ 20%). This pattern confirms the utility of the partial-credit scoring mechanism: it reveals that models *do* make meaningful progress on complex tasks, even when they fall short of full completion.

Code Tasks. Code tasks show higher performance overall, consistent with LLMs’ strength in structured text generation. GPT-5.4 achieves the best average SR of 45.0%, followed by Claude-sonnet-4.6 (41.3%). Performance varies markedly across databases: **OPTIMADE** (a standardized REST query interface) yields the highest SRs (20–85%), because its query syntax is consistent and well-documented in pretraining data. **MP** is more challenging (10–25% SR) as it requires accurate use of the `mp-api` Python client with evolving API conventions. **OQMD** tasks occupy a middle ground (0–50% SR), with large performance gaps between frontier models (Claude 50%, GPT 45%) and smaller ones (Qwen-8b 0%).

The smaller Qwen models (32b, 8b) underperform substantially on code tasks despite reasonable GUI scores, suggesting that scientific API usage demands domain-specific knowledge that scales differently from general GUI interaction.

Efficiency. Appendix Table 7 reports Eff/Eff* and FTR. GUI tasks show moderate efficiency ($\approx$ 20–30% Eff) reflecting frequent exploratory actions, while code tasks reach $\approx$ 96% Eff* since each step is a complete script execution. High FTR values (40–70%) on GUI tasks reveal a persistent premature-termination failure mode across all models.

5.3 Origin Task Evaluation

Origin tasks require the agent to generate a publication-quality scientific figure from raw experimental data using OriginPro. Each task is evaluated along two axes: (1) **file existence** — verified via `exact_match`, checking that the output `.png` is saved to the specified path; and (2) **visual conformance** — assessed by a GPT-4o judge, which receives the generated figure and a natural-language description of the expected visual result (e.g., “galvanostatic long-cycle voltage-time curve with annotation ‘5 mA cm⁻²’” and returns a binary pass/fail judgment. This two-stage design distinguishes between agents that successfully run scripts versus those that additionally produce figures that are visually and scientifically correct.

To complement the binary pass/fail judgment, Figure 6 presents a multi-dimensional quality assessment of Origin-generated figures across four task types (Raman, XRD, XPS, FTIR). Nine human evaluators and four frontier LLMs (GPT-4.5, Gemini-2.5-pro, doubao-seed-1-8, qwen3-vl-235b) independently scored each figure on a 1–5 scale across three dimensions: **visual correctness** (axes, units, data representation), **aesthetic quality** (line style, color, layout), and **task completeness** (all required elements present). Our template-guided Origin agent achieves the highest scores across all dimensions, validating both the quality of the reference template scripts and the effectiveness of the evaluation framework. Human and LLM evaluators show high agreement (Pearson $r > 0.91$), confirming that the LLM-based aesthetic judge is a reliable proxy for human judgment.

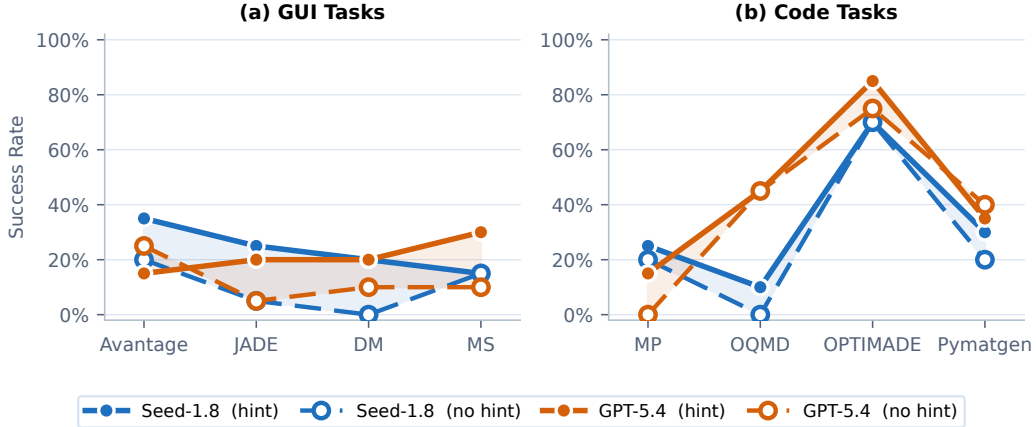


Figure 5: Ablation study: SR comparison between **hint** (solid) and **no_hint** (dashed) prompt modes for doubao-seed-1-8 (blue) and GPT-5.4 (orange). Left: GUI tasks. Right: Code tasks. Shaded bands indicate the gap between hint and no_hint conditions.

Table 3: Evaluator reliability for four GUI domains: aggregated Acc./Prec./Rec./F1 of the ground-truth getter pipeline. “Score” metrics validate value extraction; “Software” metrics validate software-state identification. Per-task breakdown is in Appendix Table 8.

Domain	Score				Software			
	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1
JADE	1.00	1.00	0.99	0.99	1.00	1.00	1.00	1.00
MS	1.00	0.99	1.00	0.99	1.00	1.00	1.00	1.00
Avantage	0.99	0.98	0.98	0.98	1.00	0.98	1.00	0.99
VESTA	0.98	0.94	0.99	0.97	0.99	0.94	1.00	0.97

5.4 Ablation Study: Effect of Prompt Templates

We ablate the effect of domain-specific prompt hints (`hint_mode`) by comparing the full agent prompt against a `no_hint` baseline that omits all domain-specific workflow instructions, evaluated on doubao-seed-1-8 and GPT-5.4.

Figure 5 summarizes the results. **Removing hints consistently degrades performance on GUI tasks:** doubao-seed-1-8 drops from 35.0% to 20.0% SR on Avantage, from 25.0% to 5.0% on JADE, and from 20.0% to 0.0% on DM. This confirms that professional software workflows benefit substantially from step-by-step operational guidance injected into the prompt.

On **code tasks**, the picture is more nuanced. For OPTIMADE, both models retain nearly the same SR with and without hints (doubao-seed-1-8: 70% $\rightarrow$ 70%; GPT-5.4: 85% $\rightarrow$ 75%), suggesting that the structured query syntax of this database is sufficiently learned by frontier models. However, on MP and OQMD, removing hints causes larger drops (GPT-5.4 MP: 15% $\rightarrow$ 0%), indicating that database-specific API conventions are less well-covered in pretraining data and benefit from explicit hint injection.

The asymmetric effect of hints across task types reflects a key property of domain-specific benchmarks: general-purpose models have uneven coverage of specialized tool ecosystems, and targeted prompt engineering can partially compensate for this knowledge gap.

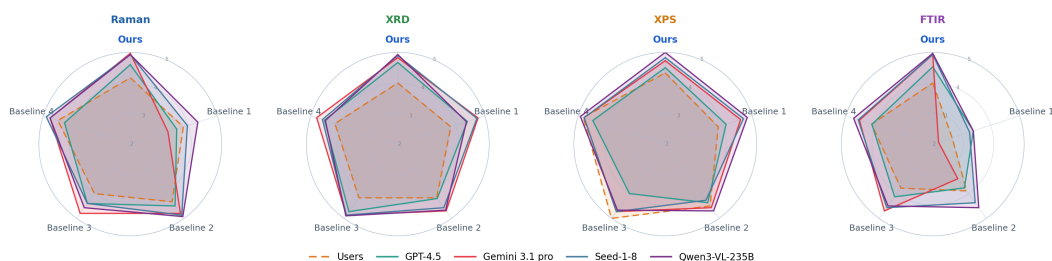


Figure 6: Radar chart comparing Origin figure quality across four task types (Raman, XRD, XPS, FTIR). Each axis scores one method (1–5 scale) on three dimensions: visual correctness, aesthetic quality, and task completeness. Evaluations by 9 human raters (dashed) and four LLMs (GPT-4.5, Gemini-2.5-pro, Seed-1.8, Qwen3-235B); Pearson $r > 0.91$ confirms LLM judges are reliable proxies for human judgment.

Acknowledgments and Disclosure of Funding

Use unnumbered first level headings for the acknowledgments. All acknowledgments go at the end of the paper before the list of references. Moreover, you are required to declare funding (financial activities supporting the submitted work) and competing interests (related financial activities outside the submitted work). More information about this disclosure can be found at: <https://neurips.cc/Conferences/2026/PaperInformation/FundingDisclosure>.

Do **not** include this section in the anonymized submission, only in the final paper. You can use the ack environment provided in the style file to automatically hide this section in the anonymized submission.

References

- Jiaxin Ai, Yukang Feng, Fanrui Zhang, Jianwen Sun, Zizhen Li, Chuanhao Li, Yifan Chang, Wenxiao Wu, Ruoxi Wang, Mingliang Zhai, Kaipeng Zhang, et al. ProSoftArena: Benchmarking hierarchical capabilities of multimodal agents in professional software environments. *arXiv preprint arXiv:2601.02399*, 2026.
- Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Buckner, Lawrence Jang, and Zack Hui. Windows Agent Arena: Evaluating multi-modal OS agents at scale. In *ICML*, 2025.
- Ruisheng Cao, Fangyu Lei, Haoyuan Wu, Jixuan Chen, Yeqiao Fu, Hongcheng Gao, Xinzhuang Xiong, Hanchong Zhang, Yuchen Mao, Wenjing Hu, Tianbao Xie, Hongshen Xu, Danyang Zhang, Sida Wang, Ruoxi Sun, Pengcheng Yin, Caiming Xiong, Ansong Ni, Qian Liu, Victor Zhong, Lu Chen, Kai Yu, and Tao Yu. Spider2-V: How far are multimodal agents from automating data science and engineering workflows? In *NeurIPS*, 2024.
- Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *ICLR*, 2025.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a generalist agent for the web. In *NeurIPS*, 2023.
- Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxuan Zhang, Junhui Li, et al. CogAgent: A visual language model for GUI agents. *CVPR*, 2024.
- Jon M. Laurent, Joseph D. Janizek, Michael Ruzo, Michaela M. Hinks, Michael J. Hammerling, Siddharth Narayanan, Manvitha Ponnampati, Andrew D. White, and Samuel G. Rodrigues. LAB-Bench: Measuring capabilities of language models for biology research. *arXiv preprint arXiv:2407.10362*, 2024.
- Siyu Liu, Bo Hu, Beilin Ye, Jiamin Xu, David J. Srolovitz, and Tongqi Wen. MatTools: Benchmarking large language models for materials science tools. *arXiv preprint arXiv:2505.10852*, 2025.
- Yadong Lu, Jianwei Li, Yelong Zhang, Wenhao Yu, Dinei Florencio, Cha Zhang, Michael Zeng Zhao, and Furu Wei. OmniParser for pure vision based GUI agent. In *CVPR*, 2025.
- OpenAI. GPT-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.
- Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Android in the wild: A large-scale dataset for android device control. In *NeurIPS*, 2023.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *ICLR*, 2021.

- Qiushi Sun, Zhoumianze Liu, Chang Ma, Zichen Ding, Fangzhi Xu, Zhangyue Yin, Haiteng Zhao, Zhenyu Wu, Kanzhi Cheng, Zhaoyang Liu, Jianing Wang, Qintong Li, Xiangru Tang, Tianbao Xie, Xiachong Feng, Xiang Li, Ben Kao, Wenhai Wang, Biqing Qi, Lingpeng Kong, and Zhiyong Wu. ScienceBoard: Evaluating multimodal autonomous agents in realistic scientific workflows. In *ICLR*, 2026.
- Minyang Tian, Luyu Gao, Shizhuo Dylan Zhang, Xinan Chen, Cunwei Fan, Xuefei Guo, Roland Haas, Pan Ji, Kittithat Krongchon, Yao Li, Shengyan Liu, Di Luo, Yutao Ma, Hao Tong, Kha Trinh, Chenyu Tian, Zihan Wang, Bohao Wu, Yanyu Xiong, Shengzhu Yin, Minhui Zhu, Kilian Lieret, Yanxin Lu, Genglin Liu, Yufeng Du, Tianhua Tao, Ofir Press, Jamie Callan, Eliu Huerta, and Hao Peng. SciCode: A research coding benchmark curated by scientists. In *NeurIPS*, 2024.
- Xiaoxuan Wang, Ziniu Hu, Pan Lu, Yanqiao Zhu, Jieyu Zhang, Satyen Subramaniam, Arjun R. Loomba, Shichang Zhang, Yizhou Sun, and Wei Wang. SciBench: Evaluating college-level scientific problem-solving abilities of large language models. In *ICML*, 2024.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Gao, Shuyu Lu, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *NeurIPS*, 2024.
- Junkai Zhang, Jingru Gan, Xiaoxuan Wang, Zian Jia, Changquan Gu, Jianpeng Chen, Yanqiao Zhu, Mingyu Derek Ma, Dawei Zhou, Ling Li, and Wei Wang. MatSciBench: Benchmarking the reasoning ability of large language models in materials science. *arXiv preprint arXiv:2510.12171*, 2025.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *ICLR*, 2024.

A Action Space

Appendix: Evaluation Protocol Details

Appendix: Efficiency Results

Table 4: Complete action interface of the Computer object executed on the Windows VM. Each call is transmitted as a Python code string via HTTP POST and evaluated with `exec(code, {"computer": computer})` on the VM server. [†] `drag` accepts raw screen pixel coordinates, unlike `move_abs`. [‡] `write` uses `pyautogui.write()` which is ASCII-only; use `clipboard.copy_text()` for any multi-line or Unicode content.

Module	Method	Parameters	Description
mouse	<code>move_id(id)</code>	Element ID (int)	Move cursor to the bounding-box center of the SoM-annotated element with the given ID
	<code>move_abs(x, y)</code>	Normalized coords $\in [0, 1]$	Move cursor to a relative screen position; (0, 0)=top-left, (1, 1)=bottom-right
	<code>single_click()</code>	—	Left single-click at the current cursor position
	<code>double_click()</code>	—	Left double-click at the current cursor position
	<code>right_click()</code>	—	Right-click at the current cursor position
	<code>scroll(dir)</code>	"up" / "down"	Scroll the active window by 400 units in the given direction
	<code>drag(x, y)[†]</code>	Screen pixel coords (int)	Left-button drag from current position to the target screen coordinates
keyboard	<code>write(text)[‡]</code>	ASCII string	Type text character-by-character via <code>pyautogui.write()</code> ; suitable only for short labels and filenames
	<code>press(key)</code>	Key name or combo	Press a single key (e.g. "f5", "enter") or hotkey combo (e.g. "ctrl+a", "alt+4")
clipboard	<code>copy_text(text)</code>	Unicode string (optional)	Copy <code>text</code> to the system clipboard via <code>pyperclip.copy()</code> ; supports full Unicode. If called with no argument, sends Ctrl+C to copy the current selection
	<code>paste()</code>	—	Paste clipboard content at the current focus via Ctrl+V
	<code>view_clipboard()</code>	—	Return the current clipboard text as a Python string (via <code>pyperclip.paste()</code>)
	<code>copy_image(id)</code>	Element ID (int)	Crop the SoM element's bounding box from the screenshot and write it to the clipboard as a DIB bitmap via <code>win32clipboard</code>
os	<code>open_program(name)</code>	Executable name (str)	Launch the named program via <code>start</code> , then maximize and bring to foreground; no-op if not installed
	<code>maximize_window()</code>	Window handle (opt.)	Maximize the specified window, or the current foreground window if omitted, via <code>win32gui</code>
	<code>is_installed(name)</code>	Executable name (str)	Return <code>True</code> if the program is found in the Windows registry or PATH
window_manager	<code>switch_to_application(name)</code>	App name (str)	Restore and maximize the closest-matching visible window (fuzzy match via <code>difflib</code>), then bring it to the foreground
	<code>find_open_applications()</code>	—	Return a list of title strings for all currently visible, non-empty windows

Table 5: Metric functions used per domain. ✓: all tasks use this metric; fractions indicate partial usage.

Domain	exact_match	detect_kv_match	detect_file_match
Avantage	✓	—	—
DM	✓	—	—
JADE	✓	—	—
MS	✓	—	—
VESTA	✓	—	—
Origin	✓	—	—
MP	—	14/20	6/20
OQMD	—	✓	—
OPTIMADE	✓	—	—
Pymatgen	—	—	✓

exact_match: binary $\{0, 1\}$ outcome for discrete GUI results and binary conditions. *detect_kv_match*: parses key-value pairs and matches numeric values within $r_{\text{tol}} = 0.1\%$, returning partial credit. *detect_file_match*: line-by-line comparison against a gold-standard reference file, returning a binary outcome.

Table 6: Getter types used in GUI-based domains, all paired with `exact_match`.

Domain	Getter Type	Description
Avantage	<code>check_multiple_files_imported</code>	Confirms that multiple XPS scan channels (<i>e.g.</i> , C 1s, O 1s, Zn 2p) are loaded.
	<code>check_dialog_exists</code>	Detects the presence of a specific dialog window via OCR.
	<code>check_dialog_parameter</code>	Extracts and validates a parameter value inside a dialog.
	<code>check_stacked_graph</code>	Verifies that spectra are displayed in stacked mode.
	<code>check_background_added</code>	Confirms background curve addition to a spectrum.
DM	<code>check_energy_axis_reversed</code>	Validates that the binding-energy axis is inverted.
	<code>check_file_import</code>	Verifies that a <code>.dm3/.dm4</code> file is imported.
	<code>check_ocr_text</code>	Extracts arbitrary text from the UI via OCR and matches it against an expected string.
	<code>check_drawing_tool</code>	Confirms the active drawing tool (box, circle, line, <i>etc.</i>).
	<code>check_border_color</code>	Detects the border color of a drawn shape via pixel analysis.
JADE	<code>check_roi_copy_and_enlarge</code>	Verifies ROI duplication and enlargement.
	<code>check_checkbox_selected</code>	Confirms a checkbox state from the accessibility tree.
	<code>check_file_opened_jade</code>	Confirms that the target XRD data file is open.
	<code>check_peak_finding</code>	Counts the number of detected peaks in the XRD pattern.
	<code>check_background_removal</code>	Validates background subtraction by detecting a $\geq 20\%$ drop in $I_{\max}$.
MS	<code>check_smoothing</code>	Confirms the “Smooth whole Pattern” operation.
	<code>check_axis_switch</code>	Verifies switching between two-theta and d -scale axes.
	<code>check_wpf_refinement</code>	Extracts the R -factor from Whole Pattern Fitting.
	<code>check_refinement_result</code>	Counts Rietveld refinement profiles.
	<code>check_dialog_title_jade</code>	Matches expected dialog titles via OCR.
VESTA	<code>check_file_in_title</code>	Confirms the project filename in the window title bar.
	<code>check_text_keyword</code>	Searches for specific keywords in UI panels (<i>e.g.</i> , “3D Atomistic”).
	<code>check_dialog_title_ms</code>	Matches dialog titles with optional forbidden-keyword filters.
	<code>check_field_value</code>	Extracts and validates form-field values (<i>e.g.</i> , space group “P1”).
	<code>check_visual_similarity</code>	Computes SSIM between the current screenshot and a reference image.
Origin	<code>check_file_opened_vesta</code>	Verifies that a CIF structure file is loaded.
	<code>check_dialog_opened_vesta</code>	Confirms a dialog window is visible.
	<code>check_atom_deletion</code>	Validates that specified atoms have been removed.
	<code>check_boundary_settings</code>	Checks boundary cell display parameters.
	<code>check_style_change</code>	Verifies rendering style changes (sphere, ball-and-stick).
Origin	<code>check_lattice_plane</code>	Confirms that a lattice plane is displayed.
	<code>check_rotation_90_up</code>	Validates a 90° upward rotation of the structure.
	<code>check_polyhedral_style</code>	Confirms polyhedral representation is active.
	<code>file_exists</code>	Checks whether the output image file was created.
	<code>validate_image_with_model</code>	Encodes the output figure as base64 and queries GPT-4o to verify it against a natural-language description of the expected plot.

 Table 7: Efficiency metrics on MATTOOLBENCH. Eff: valid-action ratio over all episodes. Eff*: valid-action ratio on successful (SR > 0) episodes. FTR (%): false termination rate. **Cat.**: task category. ‘—’: data pending; ‘NaN’: no successful episodes (division undefined).

Cat.	Domain	doubao seed-1.8			kimi k2.5			Claude son.-4.6			GPT -5.4			Qwen3 235B			Qwen 32B			Qwen3 8B		
		Eff	Eff*	FTR	Eff	Eff*	FTR	Eff	Eff*	FTR	Eff	Eff*	FTR	Eff	Eff*	FTR	Eff	Eff*	FTR	Eff	Eff*	FTR
GUI	Avantage	27.8	60.3	25.0	15.3	28.0	60.0	17.9	47.8	25.0	21.6	84.4	50.0	20.6	84.0	71.4	—	—	—	10.3	15.3	66.7
	JADE	30.8	51.6	50.0	29.4	68.8	44.4	28.3	69.6	44.4	—	—	—	16.4	61.0	71.4	—	—	—	2.6	17.3	0.0
	DM	23.5	44.8	42.9	14.2	38.5	40.0	9.3	42.8	0.0	21.4	45.4	57.1	23.8	63.0	77.8	—	—	—	0.0	0.0	NaN
	MS	4.9	16.0	50.0	2.5	22.0	50.0	—	—	—	—	—	—	0.0	NaN	100.0	52.4	—	—	0.0	NaN	NaN
	VESTA	21.9	55.5	50.0	17.9	26.8	57.1	17.9	26.8	57.1	—	—	—	33.0	79.0	75.0	39.4	18.4	83.3	0.0	0.0	NaN
Avg.		21.8	45.6	43.6	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Origin	XRD	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
	XPS	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
	Avg.	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Code	Pymatgen	96.0	96.0	70.0	73.0	96.0	80.0	—	—	—	92.3	98.0	55.0	96.0	96.0	65.0	—	—	—	73.0	96.0	81.3
	MP	96.0	96.0	75.0	96.0	96.0	80.0	96.0	96.0	85.0	98.0	NaN	100.0	95.5	96.0	75.0	—	—	—	96.0	96.0	90.0
	OQMD	96.0	96.0	90.0	39.0	98.0	30.0	96.0	96.0	96.0	96.0	96.0	55.0	96.0	96.0	90.0	—	—	—	—	—	—
	OPTIMADE	49.0	—	50.0	—	—	—	96.0	96.0	96.0	96.0	96.0	63.2	—	—	—	—	—	—	—	—	—
	Avg.	86.0	96.0	71.2	69.0	96.0	61.7	—	—	—	—	—	—	96.0	96.0	76.7	—	—	—	—	—	—
Overall Avg.		—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—

Table 8: Per-task evaluator reliability across four GUI domains: Score F1 and Software F1 (Sw.F1). Cells below 1.00 are shaded. Mean F1 per domain: JADE 0.991, MS 0.994, Avantage 0.980, VESTA 0.979.

Task	JADE		MS		Avantage		VESTA	
	F1	Sw.F1	F1	Sw.F1	F1	Sw.F1	F1	Sw.F1
in1	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
in2	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
in3	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
in4	1.00	1.00	1.00	1.00	0.88	0.92	1.00	1.00
in5	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
in6	0.92	1.00	1.00	1.00	1.00	1.00	0.97	1.00
in7	1.00	1.00	0.98	1.00	0.92	1.00	1.00	1.00
in8	1.00	1.00	1.00	1.00	0.92	0.89	1.00	1.00
in9	0.94	1.00	1.00	1.00	1.00	1.00	1.00	1.00
in10	1.00	1.00	1.00	1.00	1.00	1.00	0.86	0.86
in11	0.96	1.00	0.99	1.00	1.00	1.00	1.00	1.00
in12	1.00	1.00	0.98	1.00	0.95	1.00	1.00	1.00
in13	1.00	1.00	1.00	1.00	1.00	1.00	0.89	0.86
in14	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
in15	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
in16	1.00	1.00	0.98	1.00	0.97	1.00	1.00	1.00
in17	1.00	1.00	0.98	1.00	1.00	1.00	0.98	0.95
in18	1.00	1.00	0.97	1.00	1.00	1.00	0.91	1.00
in19	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
in20	1.00	1.00	1.00	1.00	0.96	1.00	0.97	1.00
Mean	0.991	1.000	0.994	1.000	0.980	0.991	0.979	0.984